



Red Hat OpenShift AI Self-Managed 3.5

Getting started with Red Hat OpenShift AI Self-Managed

Learn how to work in an OpenShift AI environment

Table of Contents

CHAPTER 1. OVERVIEW	3
1.1. DATA SCIENCE WORKFLOW	3
1.2. ABOUT THIS GUIDE	4
1.3. GLOSSARY OF COMMON TERMS	4
CHAPTER 2. LOGGING IN TO OPENSIFT AI	8
2.1. VIEWING INSTALLED OPENSIFT AI COMPONENTS	8
CHAPTER 3. CREATING A PROJECT	10
CHAPTER 4. CREATING A WORKBENCH AND SELECTING AN IDE	11
4.1. ABOUT WORKBENCH IMAGES	11
4.2. BUILDING THE RSTUDIO SERVER WORKBENCH IMAGES	14
4.3. CREATING A WORKBENCH	17
CHAPTER 5. NEXT STEPS	22
5.1. ADDITIONAL RESOURCES	23

CHAPTER 1. OVERVIEW

Red Hat OpenShift AI is an artificial intelligence (AI) platform that provides tools to rapidly train, serve, and monitor machine learning (ML) models onsite, in the public cloud, or at the edge.

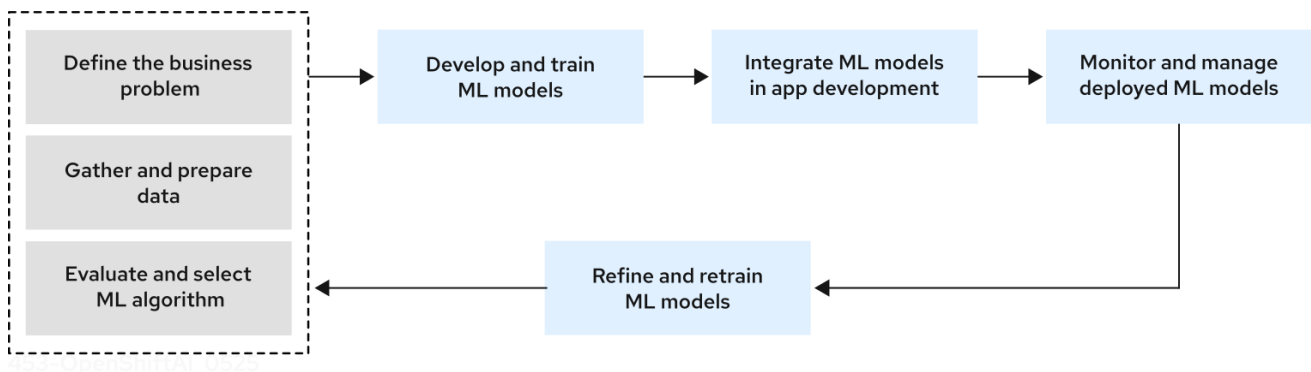
OpenShift AI provides a powerful AI/ML platform for building AI-enabled applications. Data scientists and MLOps engineers can collaborate to move from experiment to production in a consistent environment quickly.

You can deploy OpenShift AI on any supported version of OpenShift, whether on-premise, in the cloud, or in disconnected environments. For details on supported versions, see [Supported Configurations for 3.x](#).

1.1. DATA SCIENCE WORKFLOW

For the purpose of getting you started with OpenShift AI, the following figure illustrates a simplified data science workflow. The real world process of developing ML models is an iterative one.

Figure 1.1. Simplified data science workflow



The simplified data science workflow for predictive AI use cases includes the following tasks:

- Defining your business problem and setting goals to solve it.
- Gathering, cleaning, and preparing data. Data often has to be federated from a range of sources, and exploring and understanding data plays a key role in the success of a project.
- Evaluating and selecting ML models for your business use case.
- Train models for your business use case by tuning model parameters based on your set of training data. In practice, data scientists train a range of models, and compare performance while considering tradeoffs such as time and memory constraints.
- Integrate models into an application, including deployment and testing. After model training, the next step of the workflow is production. Data scientists are often responsible for putting the model in production and making it accessible so that a developer can integrate the model into an application.
- Monitor and manage deployed models. Depending on the organization, data scientists, data engineers, or ML engineers must monitor the performance of models in production, tracking prediction and performance metrics.
- Refine and retrain models. Data scientists can evaluate model performance results and refine models to improve outcome by excluding or including features, changing the training data, and modifying other configuration parameters.

1.2. ABOUT THIS GUIDE

This guide assumes you are familiar with data science and ML Ops concepts. It describes the following tasks to get you started with using OpenShift AI:

- Log in to the OpenShift AI dashboard
- Create a project
- If you have data stored in Object Storage, configure a connection to more easily access it
- Create a workbench and choose an IDE, such as JupyterLab or code-server, for your data scientist development work
- Learn where to get information about the next steps:
 - Developing and training a model
 - Automating the workflow with pipelines
 - Implementing distributed workloads
 - Testing your model
 - Deploying your model
 - Monitoring and managing your model

See also [OpenShift AI tutorial: Fraud detection example](#) . It provides step-by-step guidance for using OpenShift AI to develop and train an example model in JupyterLab, deploy the model, and refine the model by using automated pipelines.

1.3. GLOSSARY OF COMMON TERMS

This glossary defines common terms for Red Hat OpenShift AI.

accelerator

In high-performance computing, a specialized circuit that is used to take some of the computational load from the CPU, increasing the efficiency of the system. For example, in deep learning, GPU-accelerated computing is often employed to offload part of the compute workload to a GPU while the main application runs off the CPU.

artificial intelligence (AI)

The capability to acquire, process, create and apply knowledge in the form of a model to make predictions, recommendations or decisions.

AI pipelines

A workflow engine that is used by data scientists and AI engineers to automate pipelines, such as model training and evaluation pipelines. AI pipelines also includes experiment tracking capabilities, artifact storage, and versioning.

bias detection

The process of calculating fairness metrics to detect when AI models are delivering unfair outcomes based on certain attributes.

custom resource (CR)

A resource implemented through the Kubernetes CustomResourceDefinition API. A custom resource is distinct from the built-in Kubernetes resources, such as the pod and service resources. Every CR is part of an API group.

custom resource definition (CRD)

In Red Hat OpenShift, a custom resource definition (CRD) defines a new, unique object **Kind** in the cluster and lets the Kubernetes API server handle its entire lifecycle.

connections

A configuration that stores the parameters required to connect to an S3-compatible object storage, database or OCI-compliant container registry from a project.

connection type

The type of external source to connect to from a project, such as an OCI-compliant container registry, S3-compatible object storage, or Uniform Resource Identifiers (URIs).

project

An OpenShift project for organizing data science work. Each project is scoped to its own Kubernetes namespace.

disconnected environment

An environment on a restricted network that does not have an active connection to the internet.

distributed workloads

Data science workloads that are run simultaneously across multiple nodes in an OpenShift cluster.

fine-tuning

The process of adapting a pre-trained model to perform a specific task by conducting additional training. Fine tuning may involve (1) updating the model's existing parameters, known as full fine tuning, or (2) updating a subset of the model's existing parameters or adding new parameters to the model and training them while freezing the model's existing parameters, known as parameter-efficient fine tuning.

graphics processing unit (GPU)

A specialized processor designed to rapidly manipulate and alter memory to accelerate the creation of images in a frame buffer intended for output to a display. GPUs are heavily utilized in machine learning due to their parallel processing capabilities.

inference

The process of using a trained AI model to generate predictions or conclusions based on the input data provided to the model.

inference server

A server that performs inference. Inference servers feed the input requests through a machine learning model and return an output.

large language model (LLM)

A language model with a large number of parameters, trained on a large quantity of text.

machine learning (ML)

A branch of artificial intelligence (AI) and computer science that focuses on the use of data and algorithms to imitate the way that humans learn, gradually improving the accuracy of AI models.

model

In a machine learning context, a set of functions and algorithms that have been trained and tested on a data set to provide predictions or decisions.

model catalog

A curated library where data scientists can discover and evaluate the available generative AI models to find the best fit for their use cases.

model registry

A central repository containing metadata on machine learning models from inception to deployment. The metadata ranges from high-level information such as the deployment environment and project origins to specific details like training hyperparameters, performance metrics, and deployment events.

model server

A container that hosts a machine learning model, exposes an API to handle incoming requests, performs inference, and returns model predictions.

model-serving runtime

A component or framework that helps create model servers for deploying machine learning models and build APIs optimized for inference.

MLOps

The practice for collaboration between data scientists and operations professionals to help manage the production machine learning (or deep learning) lifecycle. MLOps looks to increase automation and improve the quality of production ML while also focusing on business and regulatory requirements. It involves model development, training, validation, deployment, monitoring, and management and uses methods like CI/CD.

notebook interface

An interactive document that contains executable code, descriptive text for that code, and the results of any code that is run.

object storage

A method of storing data, typically used in the cloud, in which data is stored as discrete units, or objects, in a storage pool or repository that does not use a file hierarchy but that stores all objects at the same level.

OpenShift Container Platform cluster

A group of physical machines that contains the controllers, pods, services, and configurations required to build and run containerized applications.

persistent storage

A persistent volume that retains files, models or other artifacts across components such as model deployments, AI pipelines and workbenches.

persistent volume claim (PVC)

A persistent volume claim (PVC) is a request for storage in the cluster by a user.

quantization

A method of compressing foundation model weights to speed up inferencing and reduce memory needs.

serving

The process of hosting a trained machine learning model as a network-accessible service. Real-world applications can send inference requests to the service by using a REST or gRPC API and receive predictions.

ServingRuntime

A custom resource definition (CRD) that defines the templates for pods that can serve one or more particular model formats. Each ServingRuntime CRD defines key information such as the container image of the runtime and a list of the model formats that the runtime supports. Other configuration settings for the runtime can be conveyed through environment variables in the container specification. It also dynamically loads and unloads models from disk into memory on demand and exposes a gRPC service endpoint to serve inferencing requests for loaded models.

vLLM

A high-throughput and efficient inference engine for running large-language models that integrates with popular models and frameworks.

workbench

An isolated environment for development and experimentation with ML models. Workbenches typically contain integrated development environments (IDEs), such as JupyterLab, RStudio, and Visual Studio Code.

workbench image

An image that includes preinstalled tools and libraries that you need for model development. Includes an IDE for developing your machine learning (ML) models.

YAML

A human-readable data-serialization language. It is commonly used for configuration files and in applications where data is being stored or transmitted.